



**Universidad Autónoma de Chihuahua**  
**Facultad de Ingeniería**



**Título**

2.53 Proyecto 1a. Evaluación

**Materia**

Base de Datos Avanzadas

**Docente**

José Saul De Lira Miramontes

**Alumnos**

377043 – Angel Rodriguez Palomino

377061 – Jared Isai López Espino

377304 – Zahid Alberto Jimenes Pérez

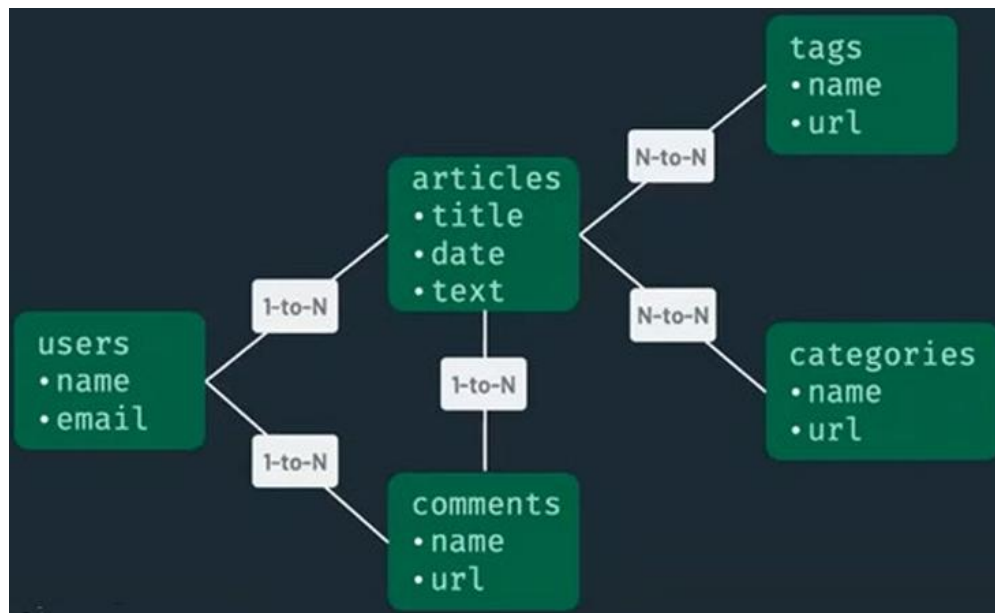
**Fecha de entrega**

25 de septiembre de 2026

## DOCUMENTACION DE APLICACIÓN EN PYTHON, UTILIZANDO FUNCIONES Y PROCEDIMIENTOS ALMACENADOS

---

Primero en base al diagrama, debemos definir las tablas que se usaran en el programa.



Para cuestiones prácticas, primero debemos eliminar o sustituir las entidades N-N, lo cual tenemos que simplemente agregar una tabla intermedia para poder realizar el ajuste correcto, esto lo hacemos debido a que los artículos pueden tener muchas categorías, y las categorías pueden pertenecer a varios artículos, simple al crear una tabla intermedia permite separar esa relación y se asegura que nosotros podamos representar esa relación N-N como una relación 1-N y N-1, esto se hace en las tablas de articles, tags y categories.

Otro ajuste más es que aparte de los atributos propuestos, se agregaron unos más, sobre todo algunos que van a ser utilizados como llaves primarias o llaves foráneas, en este caso también se debe considerar que los atributos de url no se utilizarán ya que es una aplicación de Python, lo cual no requiere de enlaces web, entonces para categories y tags le corresponde el cambio de un atributo que es llave primaria y su nombre, las tablas intermedias deberán ser compuestas ya que mencionamos que para un artículo corresponde N categorías o etiquetas, los cuales deben permanecer únicos para ese artículo.

Aquí estará el script de las tablas, tanto sus constraints que permite asegurar una mejor integridad de los datos, también por cuestiones de pruebas se puso las tablas en condición de cascada, lo cual borra toda antes de iniciar la creación de nuevo.

```
-- =====
-- SCRIPT 1: ESTRUCTURA DE BASE DE DATOS Y DATOS INICIALES
-- =====

-- 1. LIMPIEZA TOTAL

-----

DROP TABLE article_categories CASCADE CONSTRAINTS;
DROP TABLE article_tags CASCADE CONSTRAINTS;
DROP TABLE categories CASCADE CONSTRAINTS;
DROP TABLE tags CASCADE CONSTRAINTS;
DROP TABLE comments CASCADE CONSTRAINTS;
DROP TABLE articles CASCADE CONSTRAINTS;
DROP TABLE users CASCADE CONSTRAINTS;

-- 2. TABLAS

-----

CREATE TABLE users (
    user_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR2(100) NOT NULL,
    email VARCHAR2(100) NOT NULL UNIQUE,
    password VARCHAR2(100) NOT NULL,
    role VARCHAR2(20) DEFAULT 'USER' CHECK (role IN ('ADMIN', 'USER'))
);
```

User ahora tiene un id, password y role, lo cual importa mucho para la funcionalidad, esto ya que primero debemos tener un registro de los usuarios, no usamos el nombre porque existe la posibilidad de encontrar nombres repetidos, entonces se asigna de 1 en 1, la contraseña para ingresar al blog y el rol es el que determina si administra el blog (ciertas funcionalidades) o es usuario normal el cual solo puede publicar artículos.

```

CREATE TABLE articles (
    article_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id NUMBER NOT NULL,
    title VARCHAR2(150) NOT NULL,
    art_date DATE DEFAULT SYSDATE NOT NULL,
    text CLOB NOT NULL,
    CONSTRAINT fk_articles_users FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

CREATE TABLE comments (
    comment_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    article_id NUMBER NOT NULL,
    user_id NUMBER NOT NULL,
    comment_date DATE DEFAULT SYSDATE NOT NULL,
    text VARCHAR2(500) NOT NULL,
    CONSTRAINT fk_comments_articles FOREIGN KEY (article_id) REFERENCES articles(article_id) ON DELETE CASCADE,
    CONSTRAINT fk_comments_users FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

```

Tabla de artículos, tiene id, un user\_id para saber de quien es el articulo, con el fin de poder utilizar funciones que permitan consultar los artículos y desplegarlos, de explicara eso en la explicación de los procedimientos/funciones almacenadas.

Tabla comments, tiene un id, un article\_id para saber de donde son eso comentarios, user\_id para saber de quienes son esos comentarios.

```

CREATE TABLE tags (
    tag_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR2(50) NOT NULL UNIQUE
);

CREATE TABLE categories (
    category_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR2(50) NOT NULL UNIQUE
);

```

Estas tablas nos servirán para agregar nuevas categorías y etiquetas, pero estas no se utilizarán para almacenar las combinaciones.

```

CREATE TABLE article_tags (
    article_id NUMBER NOT NULL,
    tag_id NUMBER NOT NULL,
    PRIMARY KEY (article_id, tag_id),
    CONSTRAINT fk_at_articles FOREIGN KEY (article_id) REFERENCES articles(article_id) ON DELETE CASCADE,
    CONSTRAINT fk_at_tags FOREIGN KEY (tag_id) REFERENCES tags(tag_id) ON DELETE CASCADE
);

CREATE TABLE article_categories (
    article_id NUMBER NOT NULL,
    category_id NUMBER NOT NULL,
    PRIMARY KEY (article_id, category_id),
    CONSTRAINT fk_ac_articles FOREIGN KEY (article_id) REFERENCES articles(article_id) ON DELETE CASCADE,
    CONSTRAINT fk_ac_categories FOREIGN KEY (category_id) REFERENCES categories(category_id) ON DELETE CASCADE
);

```

Estas tablas son las intermedias, como se menciona antes aquí se guardarán estas combinaciones de etiquetas y categorías, lo cual significa que en estas dos tablas podremos observar un artículo corresponde N categorías o N etiquetas, al ser una llave compuesta garantizamos que nunca se presente combinaciones repetidas, ya que el `article_id` siempre es diferente y aunque `category_id` presente por ejemplo 1, 1, 2, si los artículos actuales son 1, 2, 3, los primeros dos crean una combinación única.

Ya definimos la estructura de la base de datos, ahora explicaremos los procedimientos y funciones almacenadas usadas.

```

-- SCRIPT 2: PROCEDIMIENTOS ALMACENADOS Y FUNCIONES (PL/SQL)
-- =====

-- 1. PROCEDIMIENTOS ALMACENADOS
-- -----

-- Autenticación
CREATE OR REPLACE PROCEDURE sp_login (
    p_email IN VARCHAR2,
    p_password IN VARCHAR2,
    p_user_id OUT NUMBER,
    p_name OUT VARCHAR2,
    p_role OUT VARCHAR2
) AS
    v_db_password VARCHAR2(100);
BEGIN
    SELECT user_id, name, password, role
    INTO p_user_id, p_name, v_db_password, p_role
    FROM users
    WHERE LOWER(email) = LOWER(p_email);

    IF v_db_password != p_password THEN
        RAISE_APPLICATION_ERROR(-20001, 'Contraseña incorrecta.');
```

Esta función, permite el acceso a blog, este debe recibir como parámetros el correo y contraseña, se verifica que existan en la tabla de usuario y como parámetros de salida se regresa, user\_id, nombre y rol, esto determinara que funcionalidad podrá hacer el usuario.

```

-- EXCLUSIVO ADMIN: Crear Usuario
CREATE OR REPLACE PROCEDURE sp_create_user (
    p_name IN VARCHAR2,
    p_email IN VARCHAR2,
    p_password IN VARCHAR2,
    p_role IN VARCHAR2
) AS
BEGIN
    INSERT INTO users (name, email, password, role)
    VALUES (p_name, LOWER(p_email), p_password, NVL(p_role, 'USER'));
    COMMIT;
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        RAISE_APPLICATION_ERROR(-20003, 'El correo electrónico ya existe.');
```

---

```

-- EXCLUSIVO ADMIN: Eliminar Usuario
CREATE OR REPLACE PROCEDURE sp_delete_user (
    p_user_id IN NUMBER
) AS
BEGIN
    DELETE FROM users WHERE user_id = p_user_id;
    COMMIT;
END;
```

Aquí las primeras funciones del administrador, estos podrán crear usuarios al recibir los parámetros de nombre, un correo, contraseña y el rol, mientras tanto eliminar solo requiere saber el user\_id y permite eliminarlo de forma permanente.

```

-- EXCLUSIVO ADMIN: Crear Categoría
CREATE OR REPLACE PROCEDURE sp_create_category (
    p_name IN VARCHAR2
) AS
BEGIN
    INSERT INTO categories (name) VALUES (p_name);
    COMMIT;
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        RAISE_APPLICATION_ERROR(-20004, 'La categoría ya existe.');
```

/

```

-- EXCLUSIVO ADMIN: Crear Tag
CREATE OR REPLACE PROCEDURE sp_create_tag (
    p_name IN VARCHAR2
) AS
BEGIN
    INSERT INTO tags (name) VALUES (p_name);
    COMMIT;
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        RAISE_APPLICATION_ERROR(-20005, 'La etiqueta ya existe.');
```

END;

Otra función de los administradores es la posibilidad de crear categorías, esto con el fin de poder filtrar los artículos de los usuarios de manera más fácil, mientras que las etiquetas pueden ser un adicional a los artículos.

```

-- Publicar Artículo (Disponible para USER y ADMIN)
CREATE OR REPLACE PROCEDURE sp_add_article (
    p_user_id IN NUMBER,
    p_title IN VARCHAR2,
    p_text IN VARCHAR2,
    p_article_id OUT NUMBER
) AS
BEGIN
    INSERT INTO articles (user_id, title, text)
    VALUES (p_user_id, p_title, p_text)
    RETURNING article_id INTO p_article_id;
    COMMIT;
END;
/

```

Procedimiento fundamental, este va permitir guardar los artículos escritos tanto del usuario normales y administradores, recibe el user\_id, el título, el texto y un parámetro de salida, este lo ocupamos porque requiere actualizarse el tablero principal, por eso es necesario retornar este valor tras generarlo, gracias a que se definió por default que se siguiera la enumeración básica, nunca se presenta problemas con la llave primaria.

```

-- Vincular Categorías y Tags a un Artículo
CREATE OR REPLACE PROCEDURE sp_link_article_category (p_article_id IN NUMBER, p_category_id IN NUMBER) AS
BEGIN
    INSERT INTO article_categories (article_id, category_id) VALUES (p_article_id, p_category_id);
    COMMIT;
END;
/

CREATE OR REPLACE PROCEDURE sp_link_article_tag (p_article_id IN NUMBER, p_tag_id IN NUMBER) AS
BEGIN
    INSERT INTO article_tags (article_id, tag_id) VALUES (p_article_id, p_tag_id);
    COMMIT;
END;
/

```

Estos procedimientos son los auxiliares, para las tablas intermedias siempre debemos que mantener guardado las categorías y etiquetas del articulo, para poder filtrarlos de manera correcta.

```

-- Agregar Comentario (Disponible para USER y ADMIN)
CREATE OR REPLACE PROCEDURE sp_add_comment (
    p_article_id IN NUMBER,
    p_user_id IN NUMBER,
    p_text IN VARCHAR2
) AS
BEGIN
    INSERT INTO comments (article_id, user_id, text)
    VALUES (p_article_id, p_user_id, p_text);
    COMMIT;
END;
/

```

Este procedimiento es accesible para los usuarios normales y administradores, son los comentarios que se pueden dejar debajo de los artículos, simplemente es recibir de que artículo\_id, el usuario y el texto a comentar.

```

-- EXCLUSIVO ADMIN: Eliminar Artículo
CREATE OR REPLACE PROCEDURE sp_delete_article (p_article_id IN NUMBER) AS
BEGIN
    DELETE FROM articles WHERE article_id = p_article_id;
    COMMIT;
END;
/

-- EXCLUSIVO ADMIN: Eliminar Comentario
CREATE OR REPLACE PROCEDURE sp_delete_comment (p_comment_id IN NUMBER) AS
BEGIN
    DELETE FROM comments WHERE comment_id = p_comment_id;
    COMMIT;
END;
/

```

Por cuestiones de administración, los administradores son los únicos que pueden eliminar artículos de usuarios y comentarios, esto con fines de seguridad y confiabilidad, ya que permiten a los administradores encontrar a esos usuarios que están dando mal uso de sistema de blog, lo cual permite eliminar sus cuentas o artículos como sanción.

El usuario no elimina sus artículos por esta misma razón, el administrador son los únicos autorizados de borrar tanto artículos como comentarios para mantener un orden.

Ahora de las funciones, estas tienen un objetivo muy directo, son utilizadas para consultas, ya que ocupamos que retornen pues los registros de lo que usemos.

```
CREATE OR REPLACE FUNCTION fn_get_articles RETURN SYS_REFCURSOR AS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR
    SELECT a.article_id, a.title, TO_CHAR(a.art_date, 'DD/MM/YYYY HH24:MI'), a.text, u.name
    FROM articles a
    JOIN users u ON a.user_id = u.user_id
    ORDER BY a.article_id DESC;
    RETURN v_cursor;
END;
/

CREATE OR REPLACE FUNCTION fn_get_comments (p_article_id IN NUMBER) RETURN SYS_REFCURSOR AS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR
    SELECT c.comment_id, c.text, TO_CHAR(c.comment_date, 'DD/MM/YYYY HH24:MI'), u.name
    FROM comments c
    JOIN users u ON c.user_id = u.user_id
    WHERE c.article_id = p_article_id
    ORDER BY c.comment_id ASC;
    RETURN v_cursor;
END;
/
```

Realmente es como si usáramos la operación `get()`, pero usamos un cursor `SYS_REFCURSOR` para todas las consultas, al final lo único que retornamos es ese cursor ya que en el código de Python se encarga de desplegar toda esa información, utilizando `fetchall`, en este caso esas funciones nos retornan las categorías y etiquetas del artículo que se manda.

```

CREATE OR REPLACE FUNCTION fn_get_users RETURN SYS_REFCURSOR AS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR SELECT user_id, name, email, role FROM users ORDER BY name;
    RETURN v_cursor;
END;
/

CREATE OR REPLACE FUNCTION fn_get_categories RETURN SYS_REFCURSOR AS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR SELECT category_id, name FROM categories ORDER BY name;
    RETURN v_cursor;
END;
/

CREATE OR REPLACE FUNCTION fn_get_tags RETURN SYS_REFCURSOR AS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR SELECT tag_id, name FROM tags ORDER BY name;
    RETURN v_cursor;
END;
/

```

Más funciones tipo get, muchos son utilizados para mostrar la información en la aplicación, para obtener los usuarios, en este caso sería para la funcionalidad del administrador y ver los usuarios actuales.

```

def cargar_lista_usuarios(self):
    try:
        conn = self.conectar_db()
        cursor = conn.cursor()
        ref_u = cursor.callfunc("fn_get_users", oracledb.DB_TYPE_CURSOR)

        self.lista_users_dict = {}
        opciones = []
        for u_id, u_name, u_email, u_role in ref_u.fetchall():
            label = f"{u_id} - {u_name} ({u_role})"
            opciones.append(label)
            self.lista_users_dict[label] = u_id

        ref_u.close()
        cursor.close()
        conn.close()

        if opciones:
            self.cmb_delete_user.configure(values=opciones)
            self.cmb_delete_user.set(opciones[0])
    except Exception as e:
        print(f"Error cargando usuarios: {e}")

```

```
CREATE OR REPLACE FUNCTION fn_get_categories RETURN SYS_REFCURSOR AS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR SELECT category_id, name FROM categories ORDER BY name;
    RETURN v_cursor;
END;
/

CREATE OR REPLACE FUNCTION fn_get_tags RETURN SYS_REFCURSOR AS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR SELECT tag_id, name FROM tags ORDER BY name;
    RETURN v_cursor;
END;
/
```

Más funciones para consultar, categorías y etiquetas.

```

CREATE OR REPLACE FUNCTION fn_get_article_categories (p_article_id IN NUMBER)
RETURN SYS_REFCURSOR IS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR
        SELECT c.name
        FROM categories c
        JOIN article_categories ac ON c.category_id = ac.category_id
        WHERE ac.article_id = p_article_id;

    RETURN v_cursor;
END;
/

CREATE OR REPLACE FUNCTION fn_get_article_tags (p_article_id IN NUMBER)
RETURN SYS_REFCURSOR IS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR
        SELECT t.name
        FROM tags t
        JOIN article_tags at ON t.tag_id = at.tag_id
        WHERE at.article_id = p_article_id;

    RETURN v_cursor;
END;
/

```

Estas dos son para las tablas intermedias, sirven para obtener las categorías y etiquetas de un artículo, por eso tenemos que mandar como parámetro el id.

```

# -----
# OBTENER CATEGORÍAS Y ETIQUETAS DE UN ARTÍCULO
# -----
def obtener_categorias_y_etiquetas(self, cursor, article_id):
    """Obtiene las categorías y etiquetas de un artículo mediante funciones de la BD."""
    cats, tags = [], []
    try:
        # Categorías (Llamada a función fn_get_article_categories)
        ref_cats = cursor.callfunc("fn_get_article_categories", oracledb.DB_TYPE_CURSOR, [article_id])
        cats = [r[0] for r in ref_cats.fetchall()]
        ref_cats.close()

        # Etiquetas (Llamada a función fn_get_article_tags)
        ref_tags = cursor.callfunc("fn_get_article_tags", oracledb.DB_TYPE_CURSOR, [article_id])
        tags = [r[0] for r in ref_tags.fetchall()]
        ref_tags.close()
    except Exception as e:
        print(f"Error obteniendo categorías y etiquetas del artículo {article_id}: {e}")

```

Hasta aquí hemos llegado a la parte de PL/SQL, ahora pondremos a prueba la aplicación, primero para hacerlo funcional vamos a cargar datos iniciales para realizar las ejecuciones.

```
-- 3. DATOS INICIALES
-----
-- Usuarios por defecto (ID 1 = Admin, ID 2 = Juan)
INSERT INTO users (name, email, password, role) VALUES ('Administrador Global', 'admin@blog.com', 'admin123', 'ADMIN');
INSERT INTO users (name, email, password, role) VALUES ('Juan', 'juan@blog.com', 'user123', 'USER');

-- Categorías iniciales (ID 1 = Tecnología, ID 2 = Programación)
INSERT INTO categories (name) VALUES ('Tecnología');
INSERT INTO categories (name) VALUES ('Programación');

-- Etiquetas iniciales (ID 1 = Oracle, ID 2 = Python)
INSERT INTO tags (name) VALUES ('Oracle');
INSERT INTO tags (name) VALUES ('Python');

-- Artículos iniciales
-- Artículo 1 (Escrito por el Admin)
INSERT INTO articles (user_id, title, text)
VALUES (1, 'Introducción a Oracle y Python',
'En este primer artículo aprenderemos cómo conectar Python con bases de datos Oracle siguiendo
las mejores prácticas: separando el código SQL de nuestro backend utilizando procedimientos almacenados y cursores.');
```

```
-- Artículo 2 (Escrito por Juan)
INSERT INTO articles (user_id, title, text)
VALUES (2, 'El futuro de las bases de datos', 'La tecnología avanza rápido,
pero las bases de datos relacionales siguen siendo el pilar fundamental del
entorno empresarial gracias a su robustez, seguridad y capacidad de procesamiento.');
```

```
-- Vincular artículos con sus categorías
INSERT INTO article_categories (article_id, category_id) VALUES (1, 2); -- Art 1 -> Programación
INSERT INTO article_categories (article_id, category_id) VALUES (2, 1); -- Art 2 -> Tecnología

-- Vincular artículos con sus etiquetas
INSERT INTO article_tags (article_id, tag_id) VALUES (1, 1); -- Art 1 -> Oracle
INSERT INTO article_tags (article_id, tag_id) VALUES (1, 2); -- Art 1 -> Python
INSERT INTO article_tags (article_id, tag_id) VALUES (2, 1); -- Art 2 -> Oracle

-- Comentarios iniciales
-- Juan comenta el artículo del Admin
INSERT INTO comments (article_id, user_id, text)
VALUES (1, 2, '¡Excelente artículo! Justo lo que necesitaba para estructurar correctamente mi proyecto.');
```

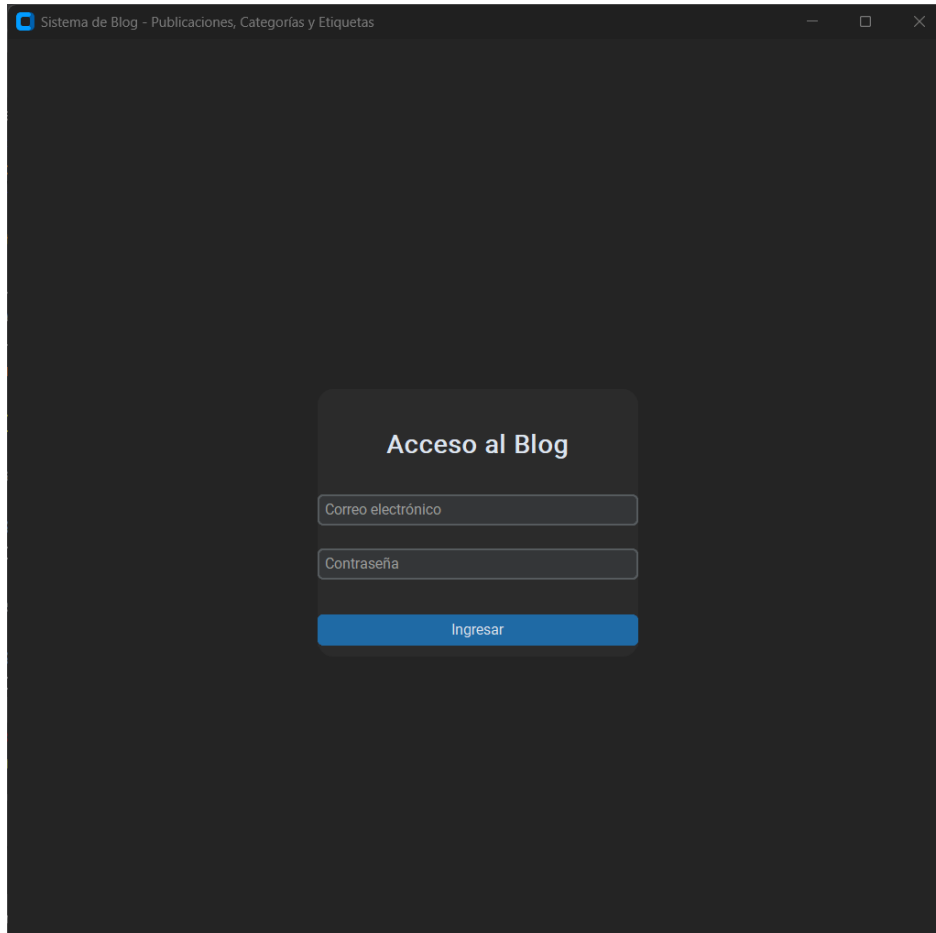
```
-- El Admin comenta el artículo de Juan
INSERT INTO comments (article_id, user_id, text)
VALUES (2, 1, 'Muy buena reflexión, Juan. Es importante conocer los fundamentos. ¡Sigue así!');

COMMIT;
```

Con estos datos iniciales podemos iniciar las pruebas, se propondrá explicar primero el apartado de administrador, con el fin de entender donde se realizan las llamadas de procedimientos y funciones.

## EJECUCIONES DEL USUARIO TIPO ADMINISTRADOR

Primero ejecutamos la aplicación, lo cual nos mostrara lo siguiente



Esta pantalla es el login al blog, para poder acceder se requiere iniciar sesión, usaremos el usuario que es admin, usamos el correo [admin@blog.com](mailto:admin@blog.com) y contraseña admin123



```
def procesar_login(self):
    email = self.ent_email.get().strip()
    password = self.ent_password.get().strip()

    if not email or not password:
        messagebox.showwarning("Atención", "Ingresa correo y contraseña.")
        return

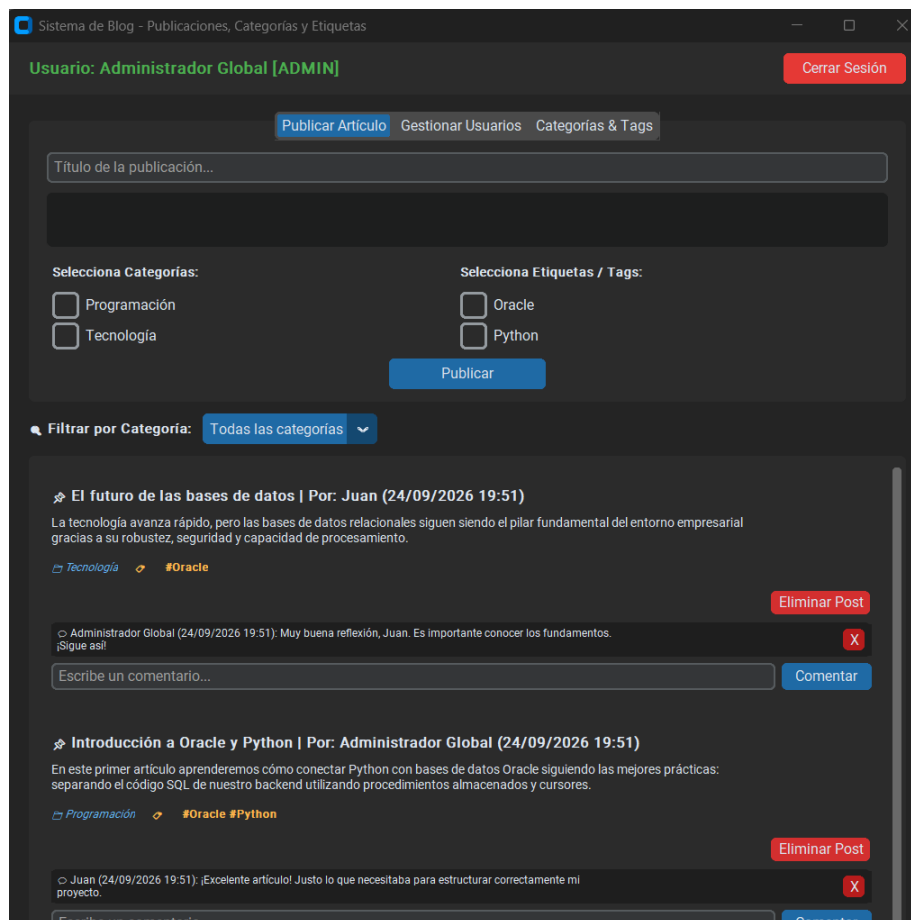
    try:
        conn = self.conectar_db()
        cursor = conn.cursor()

        user_id_out = cursor.var(oracledb.NUMBER)
        name_out = cursor.var(oracledb.STRING)
        role_out = cursor.var(oracledb.STRING)

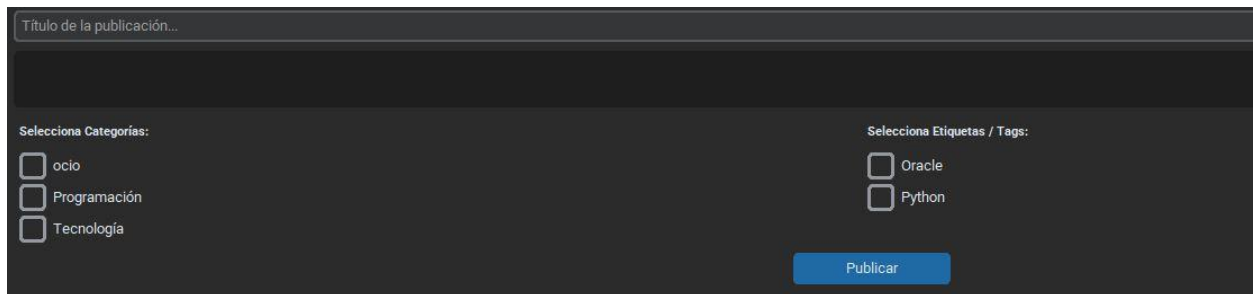
        cursor.callproc("sp_login", [email, password, user_id_out, name_out, role_out])

        self.current_user_id = int(user_id_out.getvalue())
        self.current_user_name = str(name_out.getvalue())
        self.current_user_role = str(role_out.getvalue())
```

Que debe regresar, si escribimos un usuario valido, mandamos el correo y password, si confirma que existe y es valido, nos retorna el user\_id, nombre y rol, lo cual va permitir acceso al sistema.



Se ve como mucho pero cada apartado tiene algo de importancia, para el administrador se le permite crear publicaciones al igual que usuarios normales.



Título de la publicación...

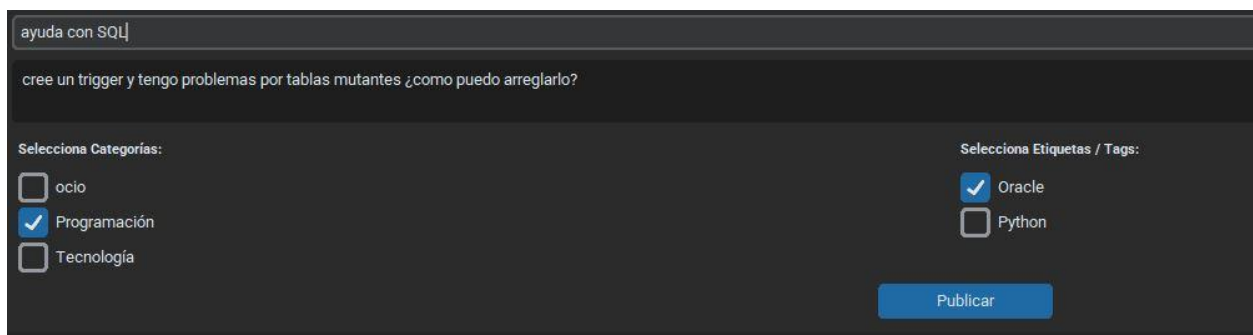
Seleccióna Categorías:

- ☐ ocio
- ☐ Programación
- ☐ Tecnología

Seleccióna Etiquetas / Tags:

- ☐ Oracle
- ☐ Python

Publicar



ayuda con SQL

creo un trigger y tengo problemas por tablas mutantes ¿como puedo arreglarlo?

Seleccióna Categorías:

- ☐ ocio
- ☒ Programación
- ☐ Tecnología

Seleccióna Etiquetas / Tags:

- ☒ Oracle
- ☐ Python

Publicar

Al publicar, esto lo que muestra:



Filtrar por Categoría: Todas las categorías

ayuda con SQL | Por: Administrador Global (24/09/2026 20:50)

creo un trigger y tengo problemas por tablas mutantes ¿como puedo arreglarlo?

Programación #Oracle

este error pasa porque intentas hacerle un SELECT, UPDATE o DELETE a la Tabla A desde un trigger que se activa justo cuando estás modificando esa misma Tabla A, una buena solución es hacer un trigger compuesto, En la sección BEFORE EACH RC

Comentar

```
try:
    conn = self.conectar_db()
    cursor = conn.cursor()
    ref_cursor = cursor.callfunc("fn_get_articles", oracledb.DB_TYPE_CURSOR)
```

Al obtener el nuevo articulo, tenemos que usar funciones get, con el fin de actualizar el tablero principal.

Esto se encuentra dentro de un ciclo, el cual usa fetchall, para obtener los registros que retorna la función al tratarse de un cursor.

```
for art_id, title, date, text, author in ref_cursor.fetchall():
    # Llamada al nuevo método renombrado
    cats, tags = self.obtener_categorias_y_etiquetas(cursor, art_id)

    # Si hay un filtro de categoría activo y el post no la contiene, lo saltamos
    if cat_id_filtro is not None and cat_seleccionada not in cats:
        continue
```

```
# Encabezado del Post
header = ctk.CTkLabel(card, text=f"🌟 {title} | Por: {author} ({date})", font=("Roboto", 13, "bold"))
header.pack(anchor="w", padx=10, pady=(8, 2))

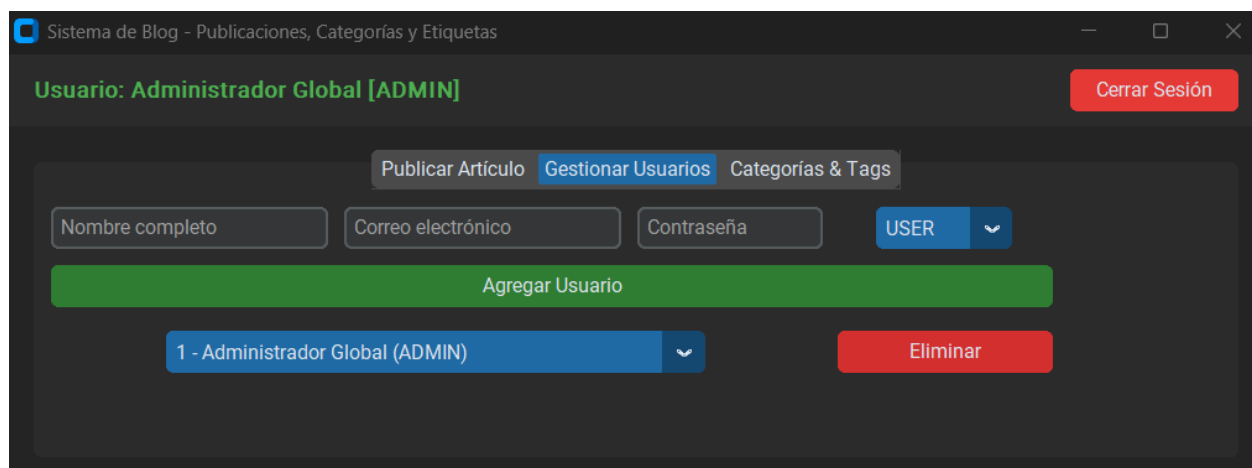
# Texto del Post
body = ctk.CTkLabel(card, text=str(text), wraplength=650, justify="left", font=("Roboto", 11))
body.pack(anchor="w", padx=10, pady=(2, 5))

# --- SECCIÓN DE CATEGORÍAS Y ETIQUETAS/TAGS EN EL POST ---
frame_etiquetas_post = ctk.CTkFrame(card, fg_color="transparent")
frame_etiquetas_post.pack(anchor="w", padx=10, pady=(0, 5))

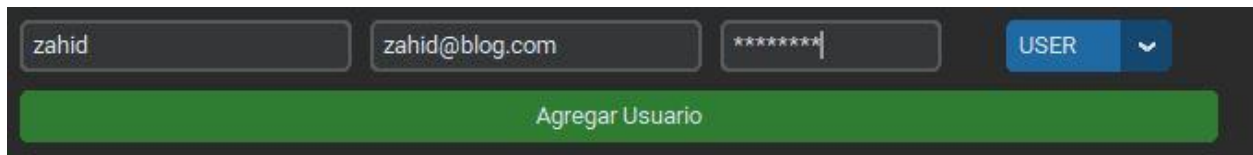
str_cats = "📁 " + ", ".join(cats) if cats else "📁 Sin Categoría"
lbl_categorias_post = ctk.CTkLabel(frame_etiquetas_post, text=str_cats, font=("Roboto", 10, "italic"), text_color="#64B5F6")
lbl_categorias_post.pack(side="left", padx=(0, 15))

str_tags = "🏷️ " + ", ".join([f"#t" for t in tags]) if tags else "🏷️ Sin Tags"
lbl_tags_post = ctk.CTkLabel(frame_etiquetas_post, text=str_tags, font=("Roboto", 10, "bold"), text_color="#FFB74D")
lbl_tags_post.pack(side="left")
```

Este apartado despliega los artículos en panel principal. La siguiente funcionalidad que puede hacer el administrador es la gestión de usuarios, como se menciono antes estos puede tanto dar de alta nuevos usuarios como dar de baja (eliminar).



Para probar dar de alta un usuario, rellenamos los campos de esta manera.



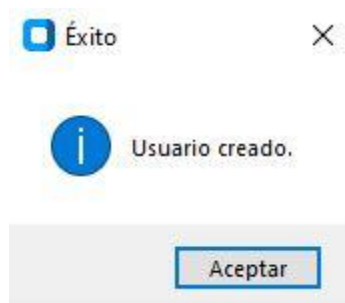
Si vemos el código, es bastante claro lo que hace:

```
def registrar_nuevo_usuario(self):
    nombre = self.ent_new_name.get().strip()
    email = self.ent_new_email.get().strip()
    password = self.ent_new_pass.get().strip()
    role = self.cmb_new_role.get()

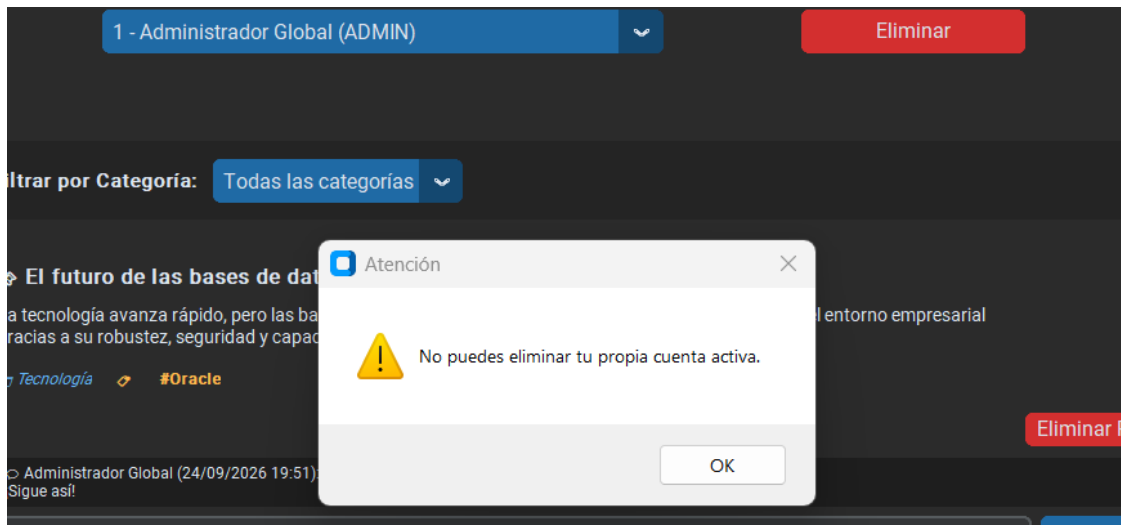
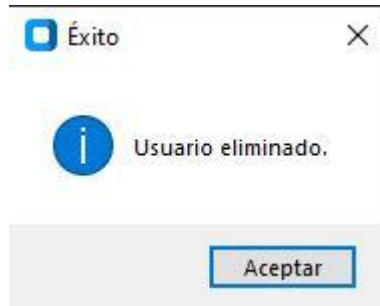
    if not nombre or not email or not password:
        messagebox.showwarning("Atención", "Campos obligatorios.")
        return

    try:
        conn = self.conectar_db()
        cursor = conn.cursor()
        cursor.callproc("sp_create_user", [nombre, email, password, role])
        cursor.close()
        conn.close()
        messagebox.showinfo("Éxito", "Usuario creado.")
        self.cargar_lista_usuarios()
    except Exception as e:
        messagebox.showerror("Error", f"{e}")
```

Obtiene la información de los campos, en dado caso de tener campos vacíos, mandar una advertencia, pero si están completos, vamos a llamar el procedimiento create\_user, el cual recibe 4 parametros y los insertara en la tabla de usuarios, se lanzara una mensaje para comunicar de la operación.

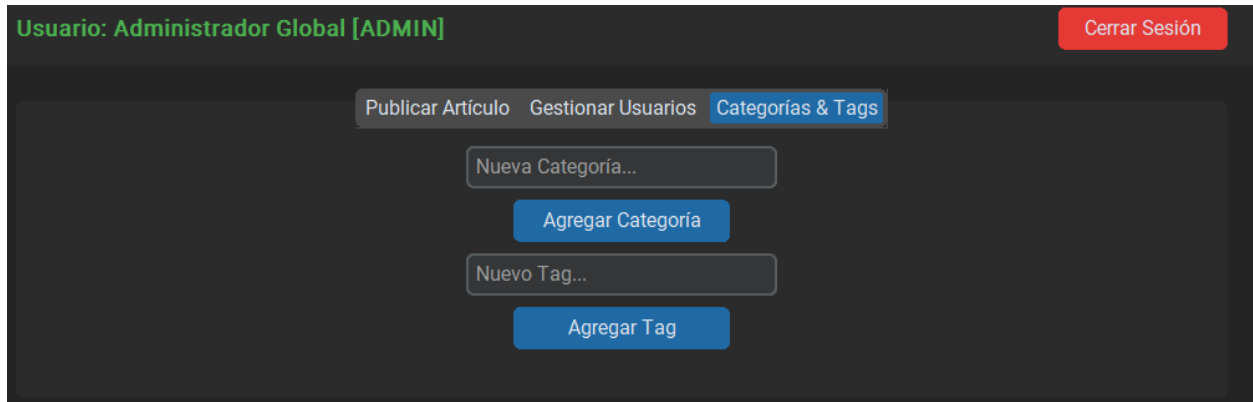


Y para eliminar, solo ocupamos saber que usuario se eliminara, aquí existe la validación para que en ningún momento el administrador pueda eliminarse a si mismo.



```
def eliminar_usuario(self):
    seleccion = self.cmb_delete_user.get()
    if seleccion in self.lista_users_dict:
        u_id = self.lista_users_dict[seleccion]
        if u_id == self.current_user_id:
            messagebox.showwarning("Atención", "No puedes eliminar tu propia cuenta activa.")
            return
        try:
            conn = self.conectar_db()
            cursor = conn.cursor()
            cursor.callproc("sp_delete_user", [u_id])
            cursor.close()
            conn.close()
            messagebox.showinfo("Éxito", "Usuario eliminado.")
            self.cargar_lista_usuarios()
        except Exception as e:
            messagebox.showerror("Error", f"{e}")
```

La siguiente funcionalidad es crear categorías y etiquetas para los usuarios.



Usuario: Administrador Global [ADMIN] Cerrar Sesión

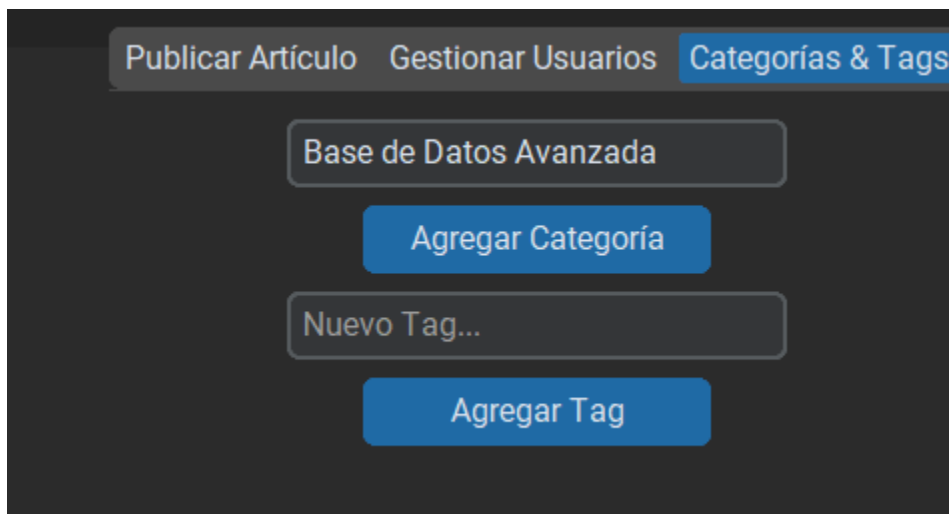
Publicar Artículo Gestionar Usuarios Categorías & Tags

Nueva Categoría...

Agregar Categoría

Nuevo Tag...

Agregar Tag



Publicar Artículo Gestionar Usuarios Categorías & Tags

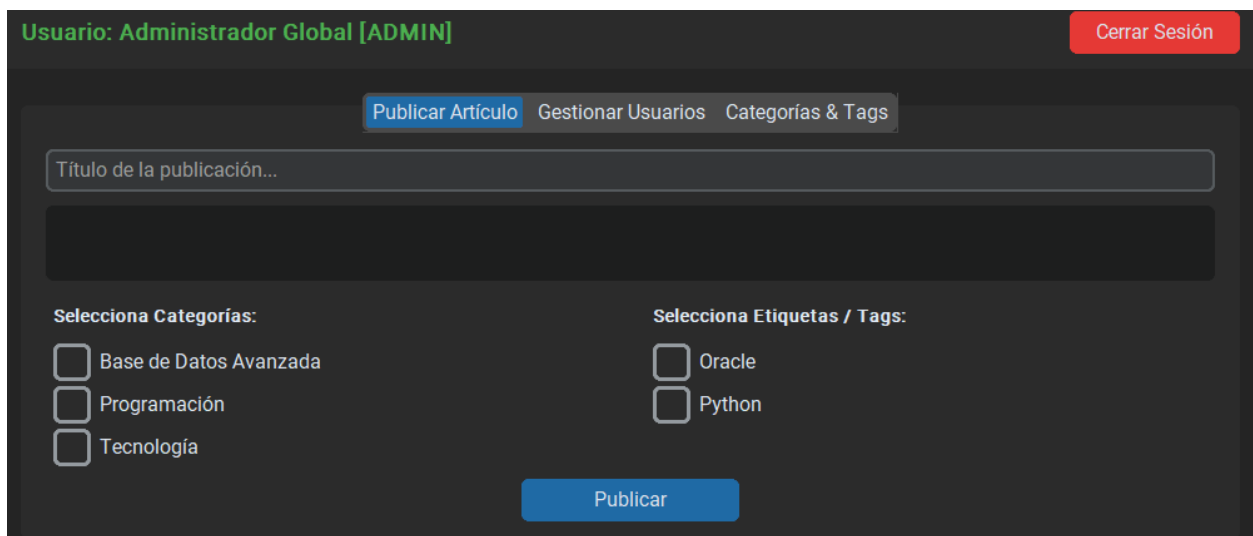
Base de Datos Avanzada

Agregar Categoría

Nuevo Tag...

Agregar Tag

Al momento de agregar categoría, sucede lo siguiente:



Usuario: Administrador Global [ADMIN] Cerrar Sesión

Publicar Artículo Gestionar Usuarios Categorías & Tags

Título de la publicación...

Selecciona Categorías:

- ☐ Base de Datos Avanzada
- ☐ Programación
- ☐ Tecnología

Selecciona Etiquetas / Tags:

- ☐ Oracle
- ☐ Python

Publicar

El código es directo, solo ocupamos llamar a el procedimiento para crear categoría o etiqueta y mandamos el nombre.

```
def agregar_categoria(self):
    cat = self.ent_new_cat.get().strip()
    if cat:
        try:
            conn = self.conectar_db()
            cursor = conn.cursor()
            cursor.callproc("sp_create_category", [cat])
            cursor.close()
            conn.close()
            self.pantalla_principal()
        except Exception as e:
            messagebox.showerror("Error", f"{e}")

def agregar_tag(self):
    tag = self.ent_new_tag.get().strip()
    if tag:
        try:
            conn = self.conectar_db()
            cursor = conn.cursor()
            cursor.callproc("sp_create_tag", [tag])
            cursor.close()
            conn.close()
            self.pantalla_principal()
        except Exception as e:
            messagebox.showerror("Error", f"{e}")
```

Ahora cuando el usuario comenta en un articulo, este debe guardar diversos datos.

ayuda con SQL | Por: Administrador Global (24/09/2026 20:50)

cree un trigger y tengo problemas por tablas mutantes ¿como puedo arreglarlo?

Programación #Oracle

zahid (24/09/2026 21:02): hola! Básicamente, este error pasa porque intentas hacerle un SELECT, UPDATE o DELETE a la Tabla A desde un trigger que se activa justo cuando estás modificando esa misma Tabla A, una buena solución es hacer un trigger compuesto. En la sección BEFORE EACH ROW, guardas los IDs o datos que necesitas en una colección (una especie de array en memoria). Luego, en la sección AFTER STATEMENT (cuando la tabla ya no está "mutando"), haces la lógica pesada usando los datos que guardaste. saludos!

```
def agregar_comentario(self, article_id, entry_widget):
    texto = entry_widget.get().strip()
    if not texto:
        return
    try:
        conn = self.conectar_db()
        cursor = conn.cursor()
        cursor.callproc("sp_add_comment", [article_id, self.current_user_id, texto])
        cursor.close()
        conn.close()
        self.cargar_articulos()
    except Exception as e:
        messagebox.showerror("Error", f"No se pudo agregar comentario: {e}")
```

Debe reconocer el id del artículo, el usuario que lo publicó y el texto, al llamar al procedimiento lo insertamos en la tabla de comments.

| Columns    | Data       | Model   | Constraints  | Grants                                                                                   | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL |
|------------|------------|---------|--------------|------------------------------------------------------------------------------------------|------------|----------|-----------|--------------|---------|------------|---------|-----|
|            | Sort...    | Filter: |              |                                                                                          |            |          |           |              |         |            |         |     |
| COMMENT_ID | ARTICLE_ID | USER_ID | COMMENT_DATE | TEXT                                                                                     |            |          |           |              |         |            |         |     |
| 1          | 1          | 1       | 24-SEP-26    | ¡Excelente artículo! Justo lo que necesitaba para estructurar correctamente mi proyecto. |            |          |           |              |         |            |         |     |
| 2          | 2          | 2       | 1 24-SEP-26  | Muy buena reflexión, Juan. Es importante conocer los fundamentos. ¡Sigue así!            |            |          |           |              |         |            |         |     |

Y para eliminar comentarios, solo tenemos que mandar el comment\_id.

🌟 El futuro de las bases de datos | Por: Juan (24/09/2026 19:51)

La tecnología avanza rápido, pero las bases de datos relacionales siguen siendo el pilar fundamental del entorno empresarial gracias a su robustez, seguridad y capacidad de procesamiento.

Tecnología #Oracle

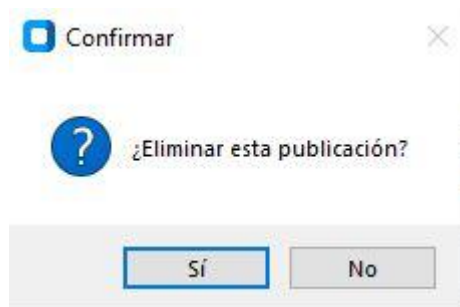
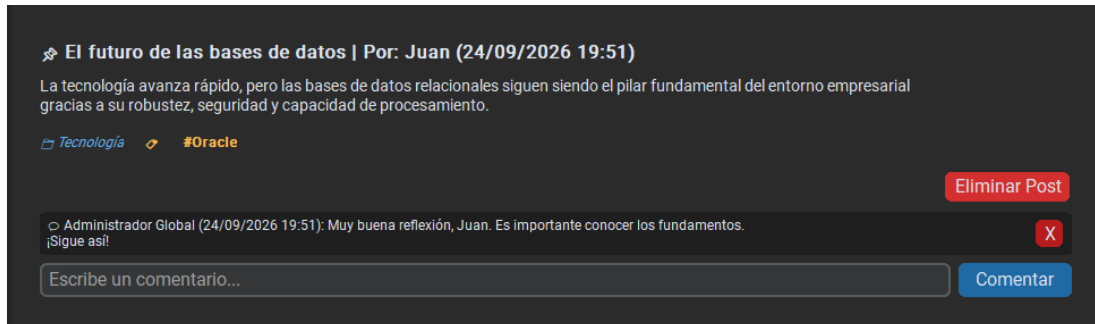
Eliminar Post

Administrador Global (24/09/2026 19:51): Muy buena reflexión, Juan. Es importante conocer los fundamentos. ¡Sigue así!

Escribe un comentario... Comentar

```
def eliminar_comentario(self, comment_id):
    try:
        conn = self.conectar_db()
        cursor = conn.cursor()
        cursor.callproc("sp_delete_comment", [comment_id])
        cursor.close()
        conn.close()
        self.cargar_articulos()
    except Exception as e:
        messagebox.showerror("Error", f"{e}")
```

Eliminar artículos al solo ser permitido por el administrador, solo requiere mandar el `article_id`, ya que esta se encuentra en cascada, todo sus hijos, los cuales son los comentarios tendrá que eliminarse también.

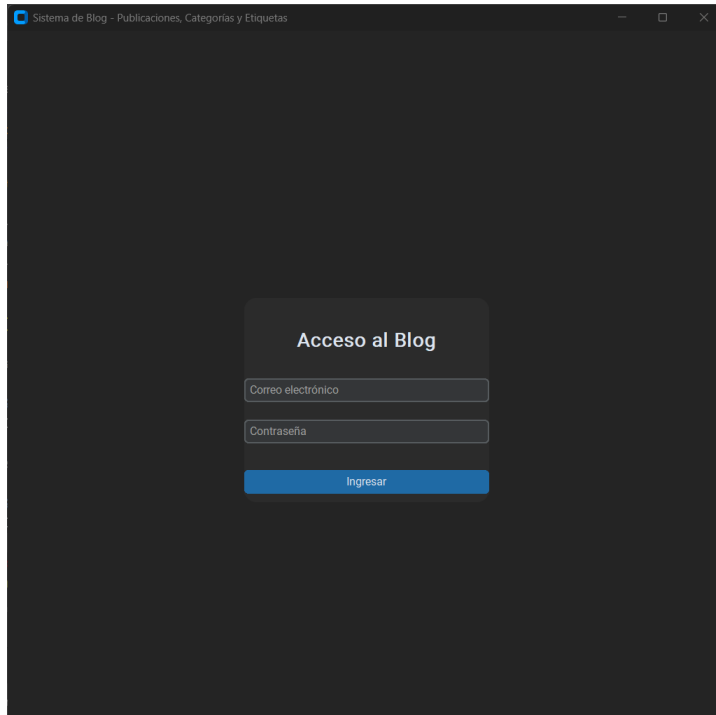


```
def eliminar_articulo(self, article_id):
    if messagebox.askyesno("Confirmar", "¿Eliminar esta publicación?"):
        try:
            conn = self.conectar_db()
            cursor = conn.cursor()
            cursor.callproc("sp_delete_article", [article_id])
            cursor.close()
            conn.close()
            self.cargar_articulos()
        except Exception as e:
            messagebox.showerror("Error", f"{e}")
```

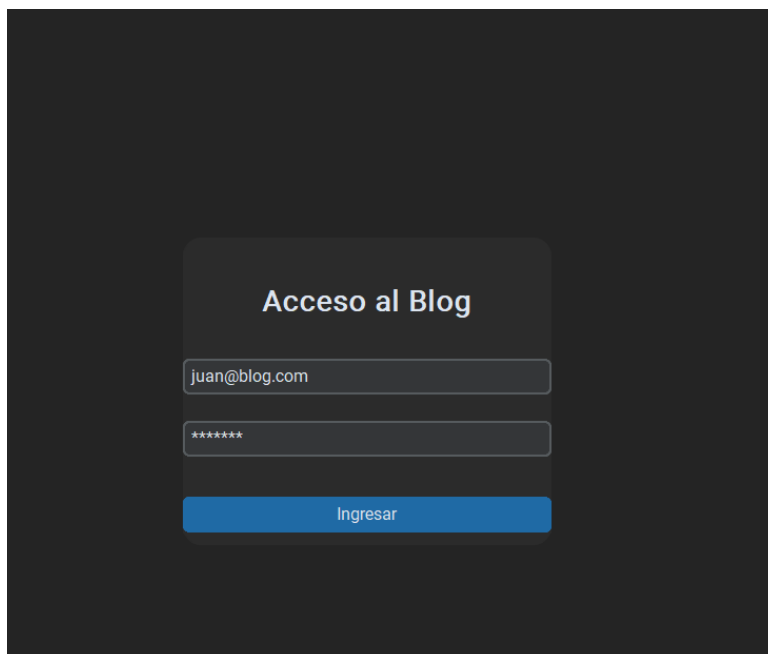
Y esto es lo que puede realizar el administrador, ahora podemos observar que puede realizar el usuario.

## EJECUCIONES DEL USUARIO TIPO NORMAL

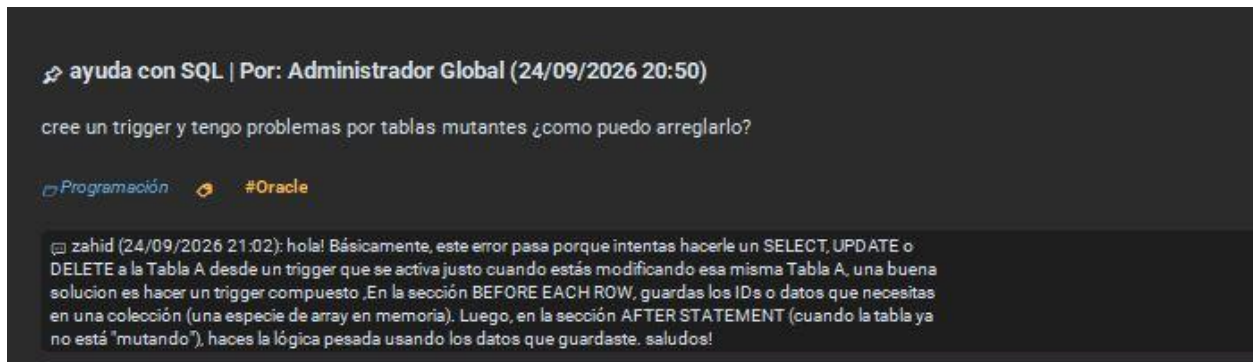
Primero ejecutamos la aplicación, lo cual nos mostrara lo siguiente



Esta pantalla es el login al blog, para poder acceder se requiere iniciar sesión, usaremos el usuario que es user, usamos el correo `juan @blog.com` y contraseña `user123`

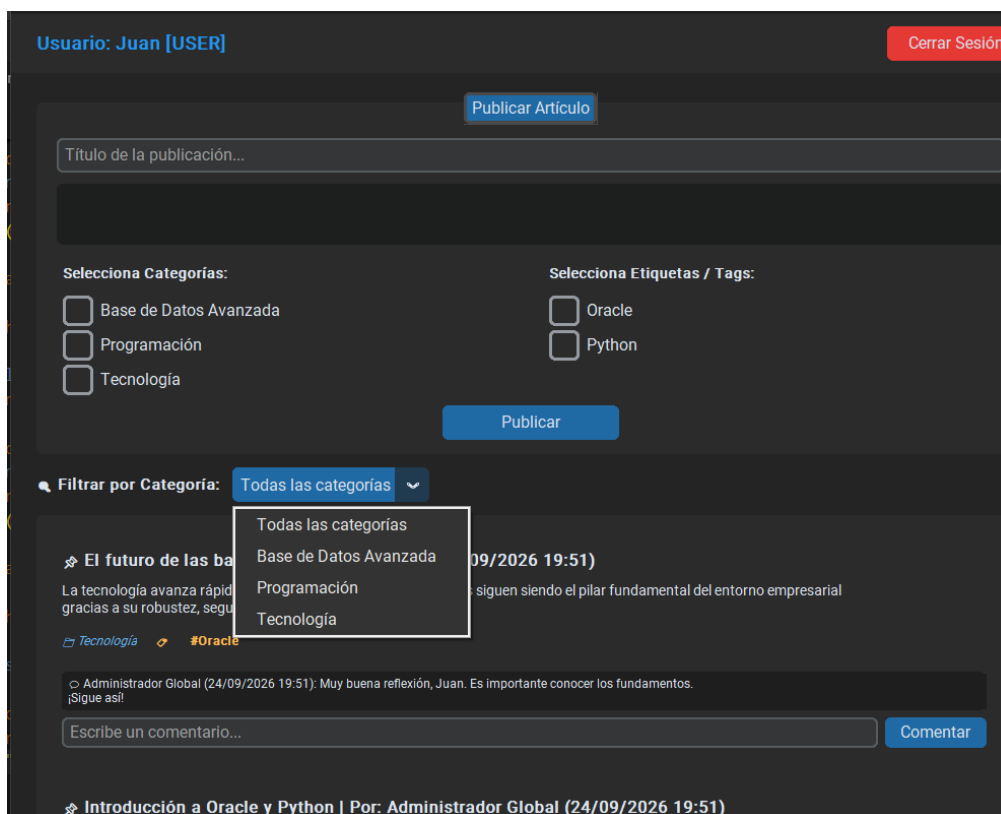


A diferencia del panel de administrador, el usuario solo se le permite “consultar” artículos, publicar artículos o comentar en artículos de otros usuarios o propios.



Es prácticamente igual que el administrador, solo que ahora cambia el usuario quien lo publica, por eso importante mandar como user\_id actual.

También se puede usar las categorías para filtrar los artículos.



Estas funciones son las que nos permiten consultar las publicaciones sobre las categorías y etiquetas.

```

# -----
# OBTENER CATEGORÍAS Y ETIQUETAS DE UN ARTÍCULO
# -----
def obtener_categorias_y_etiquetas(self, cursor, article_id):
    """Obtiene las categorías y etiquetas de un artículo mediante funciones de la BD."""
    cats, tags = [], []
    try:
        # Categorías (Llamada a función fn_get_article_categories)
        ref_cats = cursor.callfunc("fn_get_article_categories", oracledb.DB_TYPE_CURSOR, [article_id])
        cats = [r[0] for r in ref_cats.fetchall()]
        ref_cats.close()

        # Etiquetas (Llamada a función fn_get_article_tags)
        ref_tags = cursor.callfunc("fn_get_article_tags", oracledb.DB_TYPE_CURSOR, [article_id])
        tags = [r[0] for r in ref_tags.fetchall()]
        ref_tags.close()

    except Exception as e:
        print(f"Error obteniendo categorías y etiquetas del artículo {article_id}: {e}")

    return cats, tags

```

Publicar Artículo

Título de la publicación...

Selecciona Categorías:

☐ Base de Datos Avanzada
 ☐ Programación
 ☐ Tecnología

Selecciona Etiquetas / Tags:

☐ Oracle
 ☐ Python

Publicar

Filtrar por Categoría:

Programación

▼

🔖 Introducción a Oracle y Python | Por: Administrador Global (24/09/2026 19:51)

En este primer artículo aprenderemos cómo conectar Python con bases de datos Oracle siguiendo las mejores prácticas: separando el código SQL de nuestro backend utilizando procedimientos almacenados y cursores.

📁 Programación

🔖 #Oracle #Python

🗨 Juan (24/09/2026 19:51): ¡Excelente artículo! Justo lo que necesitaba para estructurar correctamente mi proyecto.

Escribe un comentario...

Comentar

Y al igual que los administradores, los usuarios puede publicar, comentar en artículos, lo que no tienen permitido es eliminar comentarios o artículos, eso quedara a disposición de los administradores.

### **Datos adicionales**

Se utiliza la librería Tkinter y para darle formato CustomTkinter, se utiliza la base de datos de Oracle, la cual tiene una conexión dedicada a solo el manejo de las tablas de blog, se contiene 7 tablas, 11 procedimientos que permite diversas operaciones y 7 funciones que sirven para consultar mediante el uso de cursores.

Se crea un repositorio para el acceso del código

Conexión utilizada:

```
DB_CONFIG = {  
    "user": "blog_user",  
    "password": "contra",  
    "dsn": "localhost/XEPDB1"  
}
```

### **ENLACE DE REPOSITORIO DE GITHUB**

<https://github.com/Magickyte/Sistema-De-Blog>